

PHISHGUARD

Detección de correos de phishing mediante Inteligencia Artificial

Ethan Provencio · Josep Maria Poblet · Jhoan Lope

Índice

01 Contexto y problemática

02 Qué hace el programa

03 Funcionalidades principales

04 Flujo del programa

05 Descripción de cada módulo

06 Algoritmia: POO y polimorfismo

07 Algoritmia: Recursividad y complejidad

08 Modelo DistilBERT y comparativa

09 Datasets utilizados

10 Resultados y conclusiones

Contexto y problemática

36%

de brechas corporativas
originan en phishing

3,4B

correos maliciosos
diarios en 2024

\$4,9M

coste medio de una
brecha de seguridad

Los sistemas clásicos basados en reglas tienen dos limitaciones críticas:

Obsolescencia rápida

Los atacantes cambian dominios y vocabulario constantemente. Una lista negra queda inútil en horas.

Alto coste de mantenimiento

Mantener miles de reglas actualizadas requiere personal especializado de forma continua.

Qué hace el programa

PHISHGUARD analiza correos electrónicos y los clasifica como Phishing o Legítimo usando DistilBERT, un modelo de inteligencia artificial preentrenado con 99.58% de precisión oficial. El sistema extrae el texto del cuerpo, los nombres de los adjuntos y las URLs de cada correo, los combina y los pasa al modelo para obtener una clasificación automática.

--eml

Correos reales (.eml)

Exportados de Gmail, Outlook o Thunderbird.
Compatible con el formato SpamAssassin.

--json

Hilo simulado (JSON)

Correos con respuestas anidadas. Demuestra la recursividad del sistema.

--csv

Dataset etiquetado (CSV)

Carga miles de correos con etiqueta real y calcula métricas de precisión.

(defecto)

Modo por defecto

Carga data/emails.json automáticamente sin necesidad de argumentos.

Funcionalidades principales

Clasificación IA

Clasifica correos como Phishing o Legítimo con DistilBERT. Muestra el porcentaje de confianza de la decisión.

Análisis recursivo

Procesa correos con respuestas anidadas de forma recursiva. Analiza cada mensaje del hilo completo.

Limpieza de HTML

Elimina automáticamente las etiquetas HTML del cuerpo antes de clasificar, mejorando la precisión.

Extracción de URLs

Detecta y extrae los enlaces del cuerpo del correo como evidencia adicional para el modelo.

Métricas de precisión

En modo CSV calcula accuracy, precision, recall, F1-score y matriz de confusión completa.

Exportación de resultados

Genera phishing.txt y legitimos.txt con asunto, remitente, confianza y texto analizado.

Soporte GPU/CPU

Detecta automáticamente la GPU NVIDIA y la utiliza. Si no hay GPU, usa la CPU sin configuración.

Modelo intercambiable

Cambiar MODEL_NAME en classifier.py actualiza todo el motor de clasificación en una línea.

Flujo del programa

ENTRADA



Descripción de cada módulo (1/2)

main.py

1. El director de orquesta

Todo empieza aquí

- Lee los argumentos: --eml, --csv, --json o modo por defecto
- Inicializa los 3 analizadores y el clasificador DistilBERT
- Llama a loader.py para cargar y analizar los correos
- Llama a reporter.py para mostrar resultados y exportar

models.py

2. La estructura de datos

Define cómo es un objeto Email

- Almacena: ID, remitente, asunto, cuerpo y adjuntos
- Contiene una lista de replies (respuestas anidadas)
- add_reply() añade una respuesta al hilo
- print_thread() muestra el hilo completo usando recursividad

email_parser.py

3. El lector de .eml

Entra en juego solo si usas --eml

- Lee archivos .eml (Gmail, Outlook, Thunderbird)
- Compatible con el formato SpamAssassin
- Extrae: remitente, asunto, cuerpo en texto plano y adjuntos
- Convierte el archivo en un objeto Email de models.py

loader.py

4. El cargador y analizador

Carga los correos y orquesta el análisis recursivo

- load_email_recursive() → construye el árbol de correos desde JSON
- Carga desde JSON, EML o CSV según el modo elegido
- Por cada correo llama a los 3 analyzers y al classifier
- analyze_thread_recursive() analiza también las respuestas anidadas

Descripción de cada módulo (2/2)

`analyzer.py`

5. Los 3 extractores (POO)

Extrae el texto relevante del correo para pasárselo al modelo

- TextAnalyzer → asunto + cuerpo sin etiquetas HTML
- AttachmentAnalyzer → nombres de los archivos adjuntos
- URLAnalyzer → enlaces encontrados en el cuerpo
- Los 3 heredan de BaseAnalyzer: polimorfismo real

`classifier.py`

6. El cerebro de IA

Recibe el texto combinado y toma la decisión

- Carga DistilBERT desde HuggingFace (~270MB, solo la 1ª vez)
- Detecta GPU automáticamente (CUDA) o usa CPU
- predict() → devuelve Phishing / Legítimo + confianza
- Cambiando MODEL_NAME se actualiza todo el motor de IA

`reporter.py`

7. El generador de informes

Muestra los resultados y exporta los ficheros de salida

- print_report() → resumen: total, phishing y legítimos
- print_metrics() → accuracy, precision, recall y F1 (solo CSV)
- export_results() → escribe phishing.txt y legitimos.txt
- Cada entrada: asunto, remitente, confianza y texto analizado

Algoritmia: POO y Polimorfismo

Encapsulación

Cada módulo oculta su lógica. PhishingClassifier expone solo predict(texto). Email expone solo add_reply() y print_thread().

Herencia

BaseAnalyzer (abstracta) obliga a implementar analyze() y get_name(). Las 3 subclases tienen su propia lógica.

Polimorfismo

loader.py llama a analyze() en cada objeto sin saber su tipo concreto. Python resuelve qué método invocar en tiempo de ejecución.

```
# analyzers.py
class BaseAnalyzer(ABC):
    @abstractmethod
    def analyze(self, email) -> str:
        pass

class TextAnalyzer(BaseAnalyzer):
    def analyze(self, email) -> str:
        clean = self._strip_html(email.body)
        return f'{email.subject} {clean}'

class URLAnalyzer(BaseAnalyzer):
    def analyze(self, email) -> str:
        urls =
        URL_PATTERN.findall(email.body)
        return ' '.join(urls)

# loader.py - polimorfismo en accion
for analyzer in analyzers: # lista de
    subclases
        part = analyzer.analyze(correo) # cada
        una lo hace distinto
```

Algoritmia: Recursividad y Complejidad

```
# loader.py
def load_email_recursive(data):
    correo = Email(
        data['email_id'],
        data['headers'], ...)
    # caso base: replies vacio -> termina
    replies = data.get('replies', [])
    for reply_data in replies:
        reply = load_email_recursive(reply_data)
        correo.add_reply(reply)
    return correo
# T(n) = 3 + 2n -> O(n)

def analyze_thread_recursive(correo, ...):
    # analizar correo actual (polimorfismo)
    for analyzer in analyzers:
        analyzer_texts[...] =
analyzer.analyze(correo)
        classifier.predict(combined_text)
    # recursion en respuestas
    for reply in correo.replies:
        analyze_thread_recursive(reply, ...)
# T(n) = 6n + 6 -> O(n)
```

Complejidad (peor caso)

Función	O()	T(n)
load_email_recursive	O(n)	$T = 3 + 2n$
analyze_thread_recursive	O(n)	$T = 6n + 6$
TextAnalyzer.analyze	O(s+b)	T = 3 por campo
AttachmentAnalyzer.analyze	O(a)	$T = 2 + n$
URLAnalyzer.analyze	O(b)	$T = 1 + 1$
PhishingClassifier.predict	O(n)	$T = 10 + 4n$
print_metrics	O(n)	$T = 33 + 7n$
export_results	O(n²)	$T = 4 + 2n + n^2$

Modelo DistilBERT v2.4.1 y comparativa

Arquitectura

DistilBERT

Tarea

**Clasificación
multilabel**

Accuracy

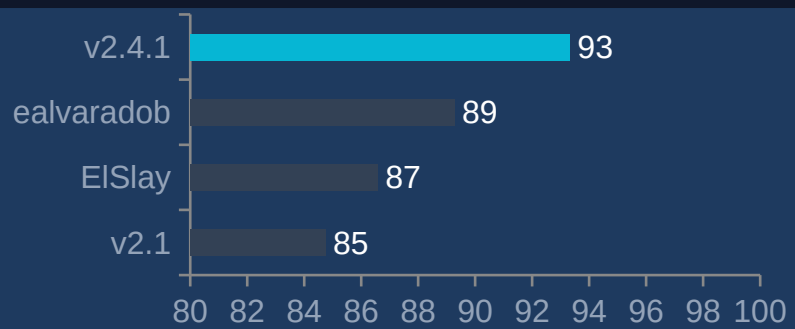
99.58%

F1-score

99.579%

Tamaño

~270 MB



¿Por qué v2.4.1?

- Mejor F1 con solo 270MB — mejor ratio rendimiento / peso
- Motor intercambiable: cambiar MODEL_NAME actualiza todo
- No requiere reentrenamiento: preentrenado con correos reales
- Detecta GPU automáticamente (CUDA) o usa CPU si no hay

Datasets utilizados

CEAS_08

2008

Total: 39.154

Phishing: 21.842 (55.8%)

Legítimos: 17.312 (44.2%)

Accuracy: **85.65%**

F1-score: **86.39%**

kaggle.com/datasets/doryanay/ceas-08

Kaggle 100k

2025

Total: 100.000

Phishing: 50.000 (50%)

Legítimos: 50.000 (50%)

Accuracy: **93.33%**

F1-score: **93.33%**

[kaggle.com/datasets/isuranga/...](https://kaggle.com/datasets/isuranga/)

SpamAssassin

2003

Total: 3.001 (.eml)

Phishing: 494/501 spam det.

Legítimos: 2471/2500 ham det.

Accuracy: **~98%**

F1-score: —

spamassassin.apache.org

Resultados de ejecución

Kaggle 100k — 2025

93.33%

Accuracy

93.34%

Precision

93.33%

Recall

93.33%

F1-score

CEAS_08 — 2008

85.65%

Accuracy

91.69%

Precision

81.67%

Recall

86.39%

F1-score

SpamAssassin (.eml)

Spam: 494 / 501 detectados (98.6%) · Ham: 2471 / 2500 correctos (98.8%)

La diferencia entre datasets refleja el año de los datos: los correos de 2008 usan técnicas más antiguas que el modelo reconoce con menor precisión. El dataset de 2025 es más representativo del phishing actual.

Conclusiones

Sistema funcional

PHISHGUARD clasifica correos reales con un 93.33% de accuracy sobre 100k correos de 2025.

Arquitectura modular

7 módulos con responsabilidades separadas. POO, polimorfismo y recursividad aplicados de forma real.

Modelo intercambiable

El motor de IA se actualiza cambiando una línea. Se probaron 4 modelos distintos durante el desarrollo.

Limitaciones conocidas

Los datasets de 2003-2008 bajan la precisión a ~85%. El modelo reconoce mejor correos modernos.